

Product Requirements Document (PRD)

Project Name: DoomOS (KDE Plasma Edition)

Document Version: 1.0.0

Status: Approved for Architecture & Design

Target Environments: Modern x86_64 Desktops, Laptops, and Handhelds

Author: Senior Systems Engineer / Technical Lead

1. Executive Overview & Mission Statement

DoomOS is a next-generation, performance-tuned Linux distribution engineered to bridge the gap between uncompromising power-user performance and an intuitive, foolproof out-of-the-box experience. Built on top of a cutting-edge **KDE Plasma 6 (Wayland-native)** desktop environment, DoomOS directly eliminates the systemic friction points that drive users away from Linux—specifically targeting:

- Gamers:** Eliminating driver breakage, Wayland frame pacing stutter, controller friction, and HDR hurdles.
- Everyday Users:** Eliminating update anxiety with automated snapshot rollbacks, blurry fractional scaling, battery drain, and codec headaches.
- Developers:** Providing seamless Wayland screen sharing with system audio, pre-configured rootless containers, and optimized kernel IPC/inotify limits.

2. Target Personas & Core Use Cases

TARGET USERS		
THE GAMER	THE EVERYDAY USER	THE DEVELOPER
High refresh, low latency	Rock-solid stability, battery	Fast toolchains, Docker,
Steam, Proton, G-Sync/HDR	Netflix, Office, App Store	Multi-monitor, Portals

Persona 1: The Modern Gamer ("Alex")

- Hardware:** Modern AMD/Intel CPU + NVIDIA RTX 3000/4000 or AMD Radeon RX 7000 series GPU, 144Hz+ VRR display.
- Goals:** Wants to install the OS, launch Steam, enable HDR, plug in an Xbox/PS5 controller, and play without editing config files or facing black screens after kernel updates.

Persona 2: The Daily Driver ("Taylor")

- Hardware:** Ultrabook / Laptop (Framework, Dell XPS, ThinkPad, ASUS Zenbook) with high-DPI display.

- **Goals:** Requires reliable suspend/resume without battery drain in backpacks, crystal-clear 125%/150% UI scaling, universal media playback, and an assurance that system updates will never brick their workstation.

Persona 3: The Full-Stack / Systems Developer ("Marcus")

- **Hardware:** Multi-monitor developer workstation or laptop, running Docker/Podman, VS Code, JetBrains IDEs, and heavy compilation workloads.
- **Goals:** Demands instant Wayland screen sharing with audio for remote meetings, high inotify file watch limits for large mono-repos, rootless container execution out-of-the-box, and isolated sandbox development environments.

3. System Architecture Specification

USERLAND INTERACTION		
KDE Plasma 6 (KWin Wayland)	Calamares Offline/Online Installer	KDE Discover / CLI
PLATFORM SUBSYSTEMS		
DoomOS Guardian (DKMS check)	Snapper / Btrfs Hooks	PipeWire Audio/Video Wire
Auto-CPUFreq / Power Profiles	xdg-desktop-portal-kde	FlatHub Sandbox Runtime
INIT & RUNTIME		
systemd (PID 1) D-Bus Message Bus Polkit ACL		
KERNEL & HARDWARE ABSTRACTION		
Linux Kernel (LTS / Zen low-latency)	Btrfs Filesystem	ZRAM Compressed Swap
Mesa Gallium/Vulkan	NVIDIA Open/DKMS Module	xone / DualSense HID Drivers

4. Non-Functional Requirements (NFRs)

4.1 Stability & Fault Tolerance (The "Immortal Boot" Guarantee)

- **Zero-Unbootable Policy:** System updates must automatically create a Btrfs pre-transaction snapshot. If an update breaks the kernel or graphics driver, the bootloader (GRUB) must expose an instant rollback menu allowing full recovery into the pre-update state in < 10 seconds.
- **Safe Driver Recompile:** NVIDIA driver DKMS compilation must execute during package installation. If the driver compilation fails, the transaction must halt before replacing the running kernel.

4.2 Performance & Responsiveness

- **Cold Boot Time:** 8 seconds from UEFI bootloader to SDDM login screen on standard NVMe PCIe Gen3+ SSDs.
- **Idle Memory Footprint:** 950 MB RAM consumption at cold boot with KDE Plasma running on Wayland.

- **Kernel Scheduling Latency:** Kernel tuned with `CONFIG_PREEMPT` / Zen scheduling patches to maintain consistent frame pacing under heavy background CPU load.
- **Memory Pressure Resilience:** Dynamic ZRAM enabled with `zstd` compression algorithm allocating 50% of physical RAM as swap space to prevent Out-Of-Memory (OOM) lockups.

4.3 Hardware Compatibility

- **Display Configurations:** Independent fractional scaling per monitor on Wayland without blur on legacy X11 applications.
- **Audio Routing:** PipeWire routing with automatic sample rate switching matching hardware capabilities without crackling or high latency.
- **Dual-Boot Interoperability:** Hardware RTC clock automatically configured to coordinate with Windows to eliminate the standard 5.5-hour time drift bug.

5. Security & Isolation Model

1. Rootless by Default:

- User account provisioned with password-less `sudo` restrictions.
- Container engines (Docker/Podman) configured for daemonless/rootless execution without requiring `sudo docker`.

2. Application Sandboxing:

- GUI applications distributed via Flathub run under Flatpak sandbox permissions managed through Plasma System Settings.

3. Immutable Boot Snapshots:

- Read-only boot snapshots mounted with copy-on-write integrity to prevent ransomware or rogue installer scripts from compromising recovery points.

6. Build & Delivery Architecture

- **Host-Agnostic Build Engine:** The ISO generation toolchain runs within an isolated container, enabling the OS to be built and verified on Linux servers, GitHub Actions CI/CD, or macOS developer workstations.
 - **Target Deliverable:** Bootable hybrid ISO (`doomos-plasma-x86_64.iso`) compatible with UEFI Secure Boot and legacy BIOS (CSM).
 - **Installer:** Fully customized **Calamares** graphical installer providing automated partitioning (Btrfs subvolumes layout `@`, `@home`, `@snapshots`, `@var_log`), timezone auto-detection, and driver selection.
-

7. Phased Release Roadmap

Phase 1: Foundation (M1)

- └─ Base Kernel + ZRAM + Btrfs Layout
- └─ KDE Plasma 6 Wayland Minimal Base
- └─ SDDM with DoomOS Theme & Autologin

Phase 2: The Core Problem Fixers (M2)

- └─ Snapper Automated Snapshot Hooks in GRUB
- └─ PipeWire Screen Sharing & Audio Loopback
- └─ Developer Sysctl tuning & Rootless Docker
- └─ NVIDIA DKMS Guardian Scripts

Phase 3: Gaming & Polishing (M3)

- └─ Wayland Explicit Sync & HDR Auto-calibrator
- └─ Game Controller (Xbox/PS5) Drivers + OpenRGB
- └─ Calamares Installer Customization & Btrfs Provisioning
- └─ Flathub Verified App Store Integration

Phase 4: ISO Mastering & Verification (M4)

- └─ Automated Build Pipeline (Dockerfile + archiso/live-build)
- └─ QEMU Virtual Machine Verification Test Suite
- └─ Golden Master ISO Release (v1.0.0)

DoomOS Technical Product Features Specification

How DoomOS Solves Every Major Linux Pain Point

This document details the exact engineering architecture, technical implementations, configuration files, and system mechanisms employed by **DoomOS** to eliminate the top pain points experienced by **Gamers**, **Everyday Users**, and **Developers**.

SECTION 1: GAMER PAIN POINTS & TECHNICAL SOLUTIONS

GAMER PROBLEM	DOOMOS ARCHITECTURAL SOLUTION
1. NVIDIA Update Black Screen	DKMS Pre-reboot Verification + Safe Rollback Hook
2. Wayland Stutter / Tearing	Wayland Explicit Sync + VRR Default in KWin
3. HDR Washed Out / Missing	Plasma 6 Tone-Mapping + Auto-Gamescope Launch Wrapper
4. Missing 32-bit Wine Libs	Comprehensive Multi-lib Baseline Pre-baked in ISO
5. Controller Connection Lag	Built-in xone / xpadneo Drivers + DualSense Haptics
6. Anti-Cheat Confusion	Real-time AreWeAntiCheatYet Badge Integration

Problem 1.1: NVIDIA Driver Updates Break the System (Black Screen on Reboot)

- Root Cause:** A new Linux kernel is installed before the NVIDIA proprietary kernel module (`nvidia.ko`) is successfully compiled by DKMS, or the driver version is incompatible with the kernel ABI. Upon reboot, the display manager (SDDM) fails to initialize DRM, stranding the user at a black screen or TTY.
- DoomOS Engineering Solution:**
 - DKMS Transaction Interceptor:** DoomOS introduces a package manager transaction hook (`/usr/share/libalpm/hooks/90-doomos-nvidia-guardian.hook`):
 - Compiles the NVIDIA kernel module *inside* the update transaction before touching `/boot/vmlinuz`.
 - Validates that `modinfo -k <new-kernel-version> nvidia` returns exit code 0.
 - If compilation fails, the entire transaction rolls back automatically and keeps the existing functional kernel active.
 - Automated Kernel Fallback:** The system always maintains both the `linux-zen` (primary performance kernel) and `linux-lts` (long-term stability fallback) side-by-side with separate boot entries.

Problem 1.2: Wayland Stutter, Micro-Jank, and Flickering

- **Root Cause:** In legacy Wayland implementations, synchronization between the compositor (KWin) and the GPU driver relied on implicit sync primitives, causing out-of-order buffer presentations, tearing, and flickering on NVIDIA hardware.
- **DoomOS Engineering Solution:**
 1. **Wayland Explicit Sync Engine:** Ships with KDE Plasma 6 + Wayland protocols (`linux-drm-syncojb-v1`) and NVIDIA driver series 555+ enabled by default.
 2. **Zero-Latency Variable Refresh Rate (VRR):** Configured in `/etc/xdg/kwinrc` :

```
[Wayland]
AllowTearingAtFullscreen=true

[Displays]
AdaptiveSync=Always
```

3. **High-Performance Memory Tuning:** Enables split-lock mitigate disabling (`split_lock_mitigate=0`) in kernel cmdline to eliminate micro-stutters during intensive game scene transitions.

Problem 1.3: HDR is Inconsistent, Washed Out, or Requires Obscure Scripts

- **Root Cause:** Upstream Linux desktop environments traditionally lacked color management infrastructure, requiring manual invocation of `gamescope` with complex color-space conversion arguments.
- **DoomOS Engineering Solution:**
 1. **Native Plasma 6 Color Pipeline:** Pre-enables KDE's native Wayland color management stack supporting HDR10 (BT.2020 PQ).
 2. **The `doom-game` One-Click Execution Wrapper:** Installs `/usr/bin/doom-game` , which auto-detects HDR-capable displays and wraps games automatically:

```
#!/usr/bin/env bash
# Auto-detects HDR monitor capability via kscreen-doctor
if kscreen-doctor -o | grep -q "HDR: supported"; then
    export ENABLE_HDR_WSI=1
    export PROTON_ENABLE_HDR=1
fi
exec gamemoderun gamescope --hdr-enabled --adaptive-sync --fullscreen -- "$@"
```

Problem 1.4: Missing 32-bit Multi-lib Libraries Crash Steam & Wine

- **Root Cause:** Users install Steam or Lutris, only to discover that ancient 32-bit game binaries fail silently because fundamental 32-bit graphics, audio, and Vulkan loaders are omitted from the default base install.
- **DoomOS Engineering Solution:**
 - The DoomOS ISO pre-bakes the complete 32-bit runtime suite:

- Graphics: `lib32-vulkan-radeon` , `lib32-nvidia-utils` , `lib32-mesa` , `lib32-vulkan-intel`
- Audio: `lib32-pipewire` , `lib32-alsa-plugins` , `lib32-libpulse`
- Crypto/Core: `lib32-gnutls` , `lib32-glibc`

Problem 1.5: Wireless Game Controllers Lag or Drop Connection

- **Root Cause:** The upstream Linux kernel lacks out-of-the-box support for the proprietary Microsoft Xbox Wireless USB Dongle protocol, and Bluetooth DualSense/Xbox pads suffer from conservative energy-saving polling intervals.
- **DoomOS Engineering Solution:**
 - 1. **Kernel Driver Modules Pre-installed:**
 - `xone-dkms` (driver for Xbox One and Series X/S wireless adapters with full audio support).
 - `xpadneo-dkms` (advanced Linux driver for Xbox One Wireless Gamepads over Bluetooth).
 - `hid-playstation` official Sony driver with full DualSense touchpad and haptic feedback support.
 - 2. **Bluetooth High-Frequency Polling:** `/etc/bluetooth/input.conf` configured with `IdleTimeout=0` and low-latency connection intervals.

SECTION 2: EVERYDAY USER PAINS & TECHNICAL SOLUTIONS

EVERYDAY USER PROBLEM	DOOMOS ARCHITECTURAL SOLUTION
1. Fear of Breaking the OS	Btrfs Snapper Automated Pre/Post Snapshots in GRUB
2. Blurry Fractional Scaling	Wayland-First Subpixel Scaling + Crisp XWayland Flags
3. Backpack Battery Drain	auto-cpufreq + Deep Sleep / Suspend Kernel Hooks
4. Headset Mono Audio Bug	PipeWire Auto-Switching Faststream / LDAC / aptX-HD
5. Missing Video Codecs	Universal GStreamer/FFmpeg + Hardware VA-API Decoding
6. App Store Chaos	Unified KDE Discover with Verified Flathub Sandbox

Problem 2.1: "Update Anxiety" (A Routine Update Renders the PC Unbootable)

- **Root Cause:** A bad update or power interruption damages libc, systemd, or kernel drivers. Non-technical users cannot fix this via chroot/TTY and reinstall Windows.
- **DoomOS Engineering Solution:**
 - 1. **Default Btrfs Subvolume Layout:**
 - `@` -> Root filesystem (`/`)
 - `@home` -> User data (`/home`) — *never rolled back, keeping user files intact*

- `@snapshots` -> System snapshots (`/.snapshots`)
- `@var_log` -> Persistent system logs (`/var/log`)

2. Automated Transaction Snapping (`snapper` + `grub-btrfs`):

- Before every update, a snapshot named `Pre-Update [timestamp]` is generated.
- After every update, a snapshot named `Post-Update [timestamp]` is generated.

3. Instant GRUB Rollback Menu:

- If the system fails to boot, the user turns on the PC, selects `DoomOS Snapshots` from the bootloader menu, picks the snapshot taken 10 minutes ago, and boots into a read-write working system instantly.

Problem 2.2: Blurry Text & Apps on Laptops (Fractional Scaling Issues)

- **Root Cause:** Under legacy X11, scaling an interface to 125% or 150% scales the display as a raster image, producing blurry fonts and window elements.
- **DoomOS Engineering Solution:**
 1. **True Wayland-Native Fractional Scaling:** KDE Plasma 6 renders UI elements at integer viewport buffers and downscales dynamically using bicubic filtering.
 2. **High-DPI Flag Injection for Legacy X11 Apps:** Automated environment profiles (`/etc/environment.d/10-hidpi.conf`):

```
# Enables crisp rendering in Qt and Electron/Chromium apps under Wayland
QT_QPA_PLATFORM="wayland;xcb"
ELECTRON_OZONE_PLATFORM_HINT="auto"
GDK_BACKEND="wayland,x11,*"
```

Problem 2.3: Laptop Overheating & Battery Drain During Sleep

- **Root Cause:** Modern laptops use ACPI "Modern Standby" (`s2idle`) instead of traditional S3 deep sleep. Linux distros often fail to disable wake-timers on USB/PCI devices, causing laptops to wake up inside backpacks and overheat.
- **DoomOS Engineering Solution:**
 1. **Kernel Sleep Policy Tuning:** `/etc/systemd/sleep.conf.d/10-doomos-power.conf` :

```
[Sleep]
MemorySleepMode=deep s2idle
HibernateDelaySec=1800
SuspendState=mem
```

2. **Intelligent Power Daemon (`auto-cpufreq`):** Monitors battery status and CPU temperature:
 - On Battery: Downclocks E-cores, throttles aggressive turbo boost, dims display refresh rate to 60Hz.
 - On AC: Unlocks full performance and high refresh rates (120Hz/144Hz/240Hz).
-

Problem 2.4: Bluetooth Audio Drops to Horrible Mono Quality During Calls

- **Root Cause:** When an application requests microphone access, PulseAudio historically switched the entire Bluetooth profile from high-fidelity A2DP (music) to low-bandwidth HSP/HFP (voice), collapsing sample rates to 8kHz mono.
- **DoomOS Engineering Solution:**
 - DoomOS ships with **PipeWire 1.2+ and WirePlumber** configured with `bluez5.enable-msbc = true` and `bluez5.enable-sbc-xq = true`.
 - When microphones are engaged, WirePlumber keeps output channels separated or utilizes mSBC wideband speech codecs, avoiding degradation of desktop background sound.

SECTION 3: DEVELOPER PAINS & TECHNICAL SOLUTIONS

DEVELOPER PROBLEM	DOOMOS ARCHITECTURAL SOLUTION
1. Broken Wayland Screen Share	xdg-desktop-portal-kde + PipeWire WebRTC Portal Hooks
2. Docker Requires `sudo`	Pre-configured Rootless Docker / Podman Group Engine
3. IDE / Webpack Inotify Crash	Enhanced /etc/sysctl.d/99-developer.conf Inotify Limits
4. Toolchain Version Clashes	Pre-installed Distrobox / BoxBuddy Containerized Dev
5. Dual-Boot Clock Drift	Automated LocalTime RTC Sync Flag in Calamares

Problem 3.1: Screen Sharing Fails or Lacks Audio in Discord, Slack, and Zoom

- **Root Cause:** Wayland does not permit applications to read pixels from the global display frame (for security). Video conferencing tools must use the Desktop Portal API (`xdg-desktop-portal`). If portals or PipeWire WebRTC hooks are missing or mismatched, screen sharing produces a black box or crashes.
- **DoomOS Engineering Solution:**
 1. **Complete Portal Suite Pre-installed:**
 - `xdg-desktop-portal-kde` (optimized for KWin Wayland)
 - `xdg-desktop-portal-gtk` (for GTK/GNOME compatibility)
 2. **Virtual Audio Loopback Sink:** Pre-configured PipeWire module allowing developers to stream their microphone *and* system sound simultaneously in Discord/Slack without third-party virtual cables:

```
# /etc/pipewire/pipewire.conf.d/10-loopback-share.conf
context.modules = [
  { name = libpipewire-module-loopback
    args = {
      node.description = "DoomOS Meeting Share Sink"
      capture.props = { media.class = "Audio/Sink" }
    }
  }
]
```

Problem 3.2: Docker Requires Root Privileges or Updates Break Network Daemon

- **Root Cause:** Traditional Docker distributions run a root daemon. Running commands without `sudo` requires manual group creation (`sudo usermod -aG docker $USER`), which is error-prone and presents security vulnerabilities.
- **DoomOS Engineering Solution:**
 1. **Rootless Docker Pre-configured:** System user accounts provisioned during Calamares installation are automatically enrolled in rootless namespaces with subuid/subgid mapping pre-populated in `/etc/subuid` and `/etc/subgid`.
 2. **Container Engine Alternatives:** Both `docker` and daemonless `podman` are installed out-of-the-box, with `podman-docker` compatibility aliases enabled.

Problem 3.3: IDEs (VS Code, Webpack, JetBrains) Crash on Large Codebases

- **Root Cause:** The Linux kernel's default `fs.inotify.max_user_watches` is set to 8,192. When an engineer opens a repository with extensive `node_modules` or thousands of source files, the watch handles are exhausted, causing silent file-saving failures or IDE crashes.
- **DoomOS Engineering Solution:**
 - Ships with high-capacity sysctl configurations in `/etc/sysctl.d/99-developer-performance.conf`:

```
# Prevent inotify exhaustion in large codebases
fs.inotify.max_user_watches = 1048576
fs.inotify.max_user_instances = 2048

# Increase memory map areas for databases and JVM
vm.max_map_count = 2147483642

# Fast network buffer tuning
net.core.somaxconn = 8192
```

Problem 3.4: Dual-Boot Clock Desynchronization with Windows

- **Root Cause:** Linux assumes the computer hardware clock (RTC) is set to UTC, while Windows assumes the hardware clock is set to Local Time. Booting between the two OSes results in the system clock jumping by several hours.

- **DoomOS Engineering Solution:**
 - The DoomOS Calamares installer automatically inspects existing partitions on all NVMe/SATA drives.
 - If a Microsoft EFI partition (`/EFI/Microsoft/Boot/bootmgfw.efi`) is detected, the installer automatically executes:

```
timedatectl set-local-rtc 1 --adjust-system-clock
```

This completely eliminates clock desynchronization out-of-the-box.

Problem 3.5: Dirty Host Toolchain Pollution (Managing Node, Python, Rust Versions)

- **Root Cause:** Developers installing multiple Python or Node versions directly on the host OS risk breaking system utilities that rely on system-level interpreters.
- **DoomOS Engineering Solution:**
 1. **Distrobox & BoxBuddy GUI Integrated:**
 - Developers can launch isolated Ubuntu, Debian, Fedora, or Alpine containers inside their host terminal with one click.
 - Full access to host files (`$HOME`), GPU acceleration, and audio without polluting host libraries.
 2. **Modern Developer Shell:**
 - Pre-configured Fish or Zsh terminal with **Starship prompt**, `fzf` fuzzy finder, `bat` (syntax-highlighted cat), and `eza` (modern ls).

SECTION 4: VERIFICATION & ACCEPTANCE CRITERIA

Feature	Automated / Manual Test Method	Pass Criteria
NVIDIA Driver Safe-Guard	Simulate broken DKMS build during mock package update.	Transaction aborts; active kernel remains untouched.
Btrfs Snapshot Rollback	Induce simulated system corruption; reboot into snapshot.	System boots to graphical desktop in \$< 10\$ seconds.
Wayland Screen Share	Launch Chromium / Discord; trigger window/screen share.	Screen captures smoothly with zero flicker or crash.
Xbox Wireless Controller	Connect Xbox dongle; verify <code>dmesg</code> output.	<code>xone</code> loads; controller pairs and operates instantly.
Inotify Limit Verification	Run watch script on 200,000 files in mock repo.	No <code>ENOSPC</code> errors thrown by kernel.
Windows Clock Sync	Install in dual-boot VM with Windows 11.	Host RTC time remains identical between OS reboots.

This document serves as the implementation blueprint for all default packages, kernel patches, and systemd configurations for DoomOS.

DoomOS Engineering Playbook

Step-by-Step Operating System Build & Implementation Guide

Document: OS Construction & Deployment Manual

Target OS: DoomOS (KDE Plasma 6 / Wayland / Btrfs)

Classification: Technical Architecture & Systems Engineering

Version: 1.0.0

Executive Overview of the Build Pipeline

Building a production-grade Linux operating system follows a deterministic, 7-phase build lifecycle. This playbook details every step required to assemble the root filesystem, inject problem-solving subsystems, brand the desktop environment, configure the installer, and master the final bootable ISO.

DOOMOS 7-PHASE BUILD LIFECYCLE	
Phase 1: Environment & Toolchain	-> Containerized Linux build engine
Phase 2: Base System Bootstrap	-> Kernel (Zen/LTS), firmware, systemd, base packages
Phase 3: Graphics & Desktop	-> SDDM, Wayland, Mesa/NVIDIA, KDE Plasma 6
Phase 4: Custom Subsystems	-> Btrfs/Snapper, DKMS Guardian, PipeWire loopback
Phase 5: Calamares Installer	-> Automated Btrfs partitioning, branding, RTC sync
Phase 6: ISO Mastering	-> Squashfs compression, GRUB hybrid UEFI/BIOS boot
Phase 7: Automated QA & Test	-> QEMU virtual test suite & bare-metal deployment

Phase 1: Build Environment & Directory Scaffolding

Because the host development machine is macOS (Apple Silicon), the build process executes within an isolated, privileged container to ensure a native, case-sensitive Linux build filesystem.

Step 1.1: Project Directory Structure Setup

The build workspace is organized into discrete modular layers:

```

doomos-builder/
├── Dockerfile           # Build engine container definition
├── build.sh             # Master one-click compilation script
├── profile/            # OS definition profiles
│   ├── packages.x86_64 # Manifest of all packages included in ISO
│   ├── pacman.conf     # Package repositories and mirrors
│   └── airootfs/        # File overlay applied to root filesystem
│       ├── etc/
│       │   ├── os-release # Distro identity and metadata
│       │   ├── sysctl.d/  # Kernel performance & inotify tuning
│       │   ├── systemd/sleep.conf.d/ # Laptop battery/suspend optimization
│       │   └── xdg/       # Default KDE Plasma 6 themes and panels
│       └── usr/
│           ├── bin/doom-game # Gaming HDR / Gamescope launch wrapper
│           └── share/
│               ├── calamares/ # Installer branding and configurations
│               └── sddm/themes/ # Login screen theme
└── output/              # Generated bootable ISO files and checksums

```

Step 1.2: Containerized Build Engine Provisioning

The builder runs via a dedicated Linux container with `CAP_SYS_ADMIN` privileges (required for `mount`, `chroot`, and `squashfs` operations):

```

# Dockerfile
FROM archlinux:base-devel
RUN pacman -Syu --noconfirm archiso git squashfs-tools dosfstools libisoburn xorriso
WORKDIR /build
ENTRYPOINT ["/build/build.sh"]

```

Phase 2: Base System Bootstrapping (Rootfs Creation)

Step 2.1: Bootstrapping the Minimal Root Filesystem

Using the package bootstrap engine (`pacstrap`), a clean root filesystem (`rootfs`) is initialized with core hardware and userspace packages:

1. Kernel Tier:

- `linux-zen` (low-latency scheduling, split-lock mitigation, optimized desktop throughput)
- `linux-lts` (long-term stability fallback kernel)
- `linux-firmware` (complete hardware firmware database for Intel, AMD, Realtek, Broadcom)

2. Core System Infrastructure:

- `systemd`, `systemd-sysvcompat`, `dbus`, `polkit`, `sudo`
- `btrfs-progs`, `e2fsprogs`, `dosfstools`, `zram-generator`
- `networkmanager`, `iw`, `bluez`, `bluez-utils`

Step 2.2: Initramfs Generation Architecture

Configure `/etc/mkinitcpio.conf` inside the rootfs:

- **Hooks enabled:** `base udev plymouth autodetect modconf kms keyboard keymap consolefont block btrfs filesystems fsck`
- Generates `vmlinux-linux-zen` and `initramfs-linux-zen.img` supporting early Plymouth boot animation and Btrfs subvolumes.

Phase 3: Graphics, Display Stack & KDE Plasma 6

Step 3.1: Wayland Compositor & Driver Stack

Install the full hardware acceleration and Wayland subsystem:

- **Graphics Drivers:** `mesa`, `vulkan-radeon`, `vulkan-intel`, `nvidia-dkms`, `nvidia-utils`
- **32-bit Compatibility Suite:** `lib32-mesa`, `lib32-vulkan-radeon`, `lib32-vulkan-intel`, `lib32-nvidia-utils`
- **Compositor Core:** `kwin`, `wayland`, `xwayland`, `qt6-wayland`

Step 3.2: Audio & Video Subsystems

- **Audio Stack:** `pipewire`, `wireplumber`, `pipewire-pulse`, `pipewire-alsa`, `pipewire-jack`
- **Multimedia Codecs:** `ffmpeg`, `gststreamer`, `gst-plugins-good`, `gst-plugins-bad`, `gst-plugins-ugly`, `gst-libav`

Step 3.3: Display Manager (SDDM) & Live Autologin

1. Enable SDDM service: `systemctl enable sddm.service`
2. Configure live session autologin in `/etc/sddm.conf.d/liveuser.conf`:

```
[Autologin]
User=liveuser
Session=plasmawayland
```

Phase 4: Implementing the Problem-Solving Subsystems

Step 4.1: Btrfs Automated Snapshot Rollback Setup

1. Install `snapper`, `snap-pac`, and `grub-btrfs`.
2. Configure `/etc/snapper/configs/root`:
 - `TIMELINE_CREATE=yes`, `NUMBER_LIMIT=10`
3. Configure `snap-pac` to take pre- and post-transaction snapshots on every update.

4. Enable `grub-btrfsd.service` to dynamically regenerate the GRUB boot menu whenever a snapshot is taken.

Step 4.2: NVIDIA DKMS Guardian Hook

Deploy `/usr/share/libalpm/hooks/90-doomos-nvidia-guardian.hook` :

```
[Trigger]
Operation = Upgrade
Operation = Install
Type = Package
Target = nvidia-dkms
Target = linux-zen

[Action]
Description = DoomOS Guardian: Verifying NVIDIA module compilation...
When = PostTransaction
Exec = /usr/bin/doomos-nvidia-verify
```

The verification script checks module integrity; if missing or broken, it issues an immediate warning and creates a recovery boot fallback.

Step 4.3: Developer Kernel & Inotify Configuration

Deploy `/etc/sysctl.d/99-developer-performance.conf` :

```
# Prevent inotify exhaustion in large codebases
fs.inotify.max_user_watches = 1048576
fs.inotify.max_user_instances = 2048

# Virtual memory mapping limits for databases / JVM
vm.max_map_count = 2147483642

# Network connection backlog
net.core.somaxconn = 8192
```

Step 4.4: Wayland Desktop Portal & Loopback Audio Setup

1. Install `xdg-desktop-portal-kde` and `xdg-desktop-portal-gtk` .
2. Inject virtual audio loopback profile in `/etc/pipewire/pipewire.conf.d/10-loopback.conf` to enable simultaneous microphone and system sound streaming.

Step 4.5: Power & Sleep Optimization

1. Install `auto-cpufreq` and `power-profiles-daemon` .
 2. Enable `auto-cpufreq.service` to intelligently scale CPU states between AC and battery.
 3. Configure `/etc/systemd/sleep.conf.d/10-doomos-sleep.conf` with `MemorySleepMode=deep s2idle` .
-

Phase 5: Calamares Graphical Installer Integration

Step 5.1: Installer Layout & Modules

Configure `/etc/calamares/settings.conf` :

- **Sequence:**
 1. `welcome` (language selection, network status)
 2. `locale` (timezone selection with automatic IP geolocation)
 3. `keyboard` (layout preview)
 4. `partition` (custom DoomOS automated Btrfs partitioning module)
 5. `users` (creation of user, host name, root password)
 6. `unpackfs` (unpacking the squashfs image to target SSD/NVMe)
 7. `doomos-rtc` (Windows dual-boot detection & RTC adjustment)
 8. `bootloader` (GRUB 2 EFI + snapshot menu generation)

Step 5.2: Dual-Boot Windows RTC Sync Script

Script injected into `/usr/lib/calamares/modules/doomos-rtc/main.py` :

```
# Checks if Windows EFI loader exists on any mounted disk
import os, subprocess

if os.path.exists("/boot/efi/EFI/Microsoft/Boot/bootmgfw.efi"):
    subprocess.run(["timedatectl", "set-local-rtc", "1", "--adjust-system-clock"])
```

Phase 6: Squashfs Compression & ISO Mastering

Step 6.1: Compacting Rootfs into Squashfs

The entire root filesystem is compressed using the high-performance Zstandard algorithm:

```
mksquashfs airootfs/ output/airootfs.sfs -comp zstd -Xcompression-level 19 -b 1M
```

Step 6.2: EFI System Partition & Bootloader Generation

1. Format EFI system partition image (`efiboot.img`).
2. Populate with GRUB 2 EFI binaries (`B00TX64.EFI`) signed for Secure Boot compatibility.
3. Generate `grub.cfg` containing live boot parameters:

```
linux /boot/vmlinuz-linux-zen archisobasedir=doomos archisolabel=D00M0S_LIVE quiet splash co
initrd /boot/initramfs-linux-zen.img
```

Step 6.3: Mastering the Hybrid ISO with Xorriso

Generate the final bootable ISO image:

```
xorriso -as mkisofs \  
  -iso-level 3 \  
  -full-iso-9660-filenames \  
  -volid "DOOMOS_LIVE" \  
  -eltorito-boot isolinux/isolinux.bin \  
  -eltorito-catalog isolinux/boot.cat \  
  -no-emul-boot -boot-load-size 4 -boot-info-table \  
  -isohybrid-mbr isolinux/isohdpx.bin \  
  -eltorito-alt-boot \  
  -e boot/grub/efiboot.img \  
  -no-emul-boot -isohybrid-gpt-basdat \  
  -output output/doomos-plasma-x86_64.iso \  
  iso_root/
```

Phase 7: Automated Testing & Deployment

Step 7.1: Virtual Machine Verification (QEMU)

Verify the newly created ISO boots to a full KDE Plasma desktop:

```
qemu-system-x86_64 \  
  -enable-kvm \  
  -m 4G \  
  -smp 4 \  
  -cpu host \  
  -bios /usr/share/ovmf/x64/OVMF.fd \  
  -cdrom output/doomos-plasma-x86_64.iso \  
  -vga virtio \  
  -device virtio-sound-pci,audiodev=audio0 \  
  -audiodev coreaudio,id=audio0
```

Step 7.2: Bare-Metal Deployment Checklist

1. Write the ISO to a USB drive:

```
sudo dd if=output/doomos-plasma-x86_64.iso of=/dev/sdX bs=4M status=progress oflag=sync
```

2. Boot test on target hardware:

- Verify Wi-Fi / Bluetooth scanning in SDDM.
 - Verify Wayland fractional scaling on 1080p, 1440p, and 4K displays.
 - Run Calamares installer and verify Btrfs subvolumes and snapshot recovery.
 - Launch Steam and verify GameMode and Xbox controller connectivity.
-

Complete Build Summary Table

Step	Action	Tools Used	Output Artifact
1	Build Scaffolding	Docker, Git	Build container environment
2	Rootfs Bootstrap	<code>pacstrap</code> , <code>mkinitcpio</code>	<code>rootfs/</code> + <code>vmlinuz-linux-zen</code>
3	Graphics & Desktop	Mesa, SDDM, KDE Plasma 6	Wayland desktop session
4	Subsystem Injection	Btrfs, Snapper, PipeWire	Auto-rollback & audio loopback
5	Installer Setup	Calamares, Python modules	Branded graphical installer
6	ISO Mastering	<code>mksquashfs</code> (<code>zstd</code>), <code>xorriso</code>	<code>doomos-plasma-x86_64.iso</code>
7	QA & Deployment	QEMU, OVMF, <code>dd</code>	Verified Golden Master OS

This playbook provides the end-to-end technical steps to construct, customize, and master DoomOS.